

Manuale del programma q_gen

Lavinia Egidi
Università del Piemonte Orientale

September 2026

Indice

1	Introduzione	2
2	Il programma	3
2.1	Per iniziare velocemente	3
2.2	Dettagli e opzioni	3
2.3	Concatenazione	5
3	Formato dei file di input	5
3.1	Chiavi per tutti i tipi di domande	5
	"categoria"	5
	"nome_domanda"	5
	"tipo_domanda"	5
	"frasi"	6
	"numero_di_frase"	6
	"scelte"	6
3.2	Chiavi per le domande "dd"	6
	"risposte"	6
	"consegna"	6
	"spazio_risposta"	6
	"se_infinite"	7
3.3	Chiavi per le domande "cloze"	7
	"tipo_di_cloze"	7
	"risposte"	7
	"consegna"	7
3.4	Chiavi per le domande "mcq"	7
	"frazioni_gruppi"	7
3.5	Dettagli sul campo "risposte"	7
3.6	Dettagli sul campo "frazioni_gruppi"	9
3.7	Dettagli sul campo "scelte"	9
3.7.1	"scelte" per "mcq"	10
3.8	Opzioni: campo "opzioni"	11

"moodle_ordine_casuale"	11
"risposte_multiple"	11
"elenco_puntato"	11
"permutazioni"	11
3.8.1 Combinazioni e "permutazioni"	11
4 File di configurazione	11
4.1 Parametri utili all'utente	12
"exec"	12
"database"	12
"filenames"	12
4.2 Altri parametri	12
"templates"	13
"necessary_input_fields"	13
"meaningful_options"	13
"cloze_types"	13
A Formato dei file JSON	13
Commenti in un file JSON	14
B Nomi delle chiavi dei file in input	14

1 Introduzione

Il programma `q_gen` permette di generare delle raccolte di domande in formato Moodle XML a partire da file di input. Nei file di input sono elencate le alternative da utilizzare e altre informazioni per la preparazione delle domande (tipo di domanda, categoria, titolo, ecc.). I file prodotti possono essere importati in Moodle.

Il programma è in grado di produrre domande dei seguenti tipi:

- a scelta multipla
- drag&drop
- cloze a scelta singola
- cloze a risposta aperta

Il programma viene distribuito senza alcuna garanzia, nella speranza che possa essere utile ad altri. Si raccomanda di verificare che le domande prodotte siano corrette prima di utilizzarle poiché potrebbero esserci degli errori nel programma o non essere chiaro il significato di alcune opzioni.

È distribuito con alcuni esempi che possono aiutare a capire il formato richiesto per i file di input senza che sia necessario leggere tutto il presente manuale.

2 Il programma

2.1 Per iniziare velocemente

Per ottenere il programma, aprire un terminale e digitare

```
git clone https://github.com/laviniaegidi-UP0/q_gen.git
```

Verrà creata, nella cartella di lavoro, una cartella `q_gen`. Spostarsi nella cartella `q_gen`. Sotto `linux` il comando è:

```
cd q_gen
```

. Nella cartella, eseguire il comando:

```
python3 q_gen.py
```

Questo avrà l'effetto di eseguire il programma sui file di esempio che sono distribuiti col codice e che si trovano nella cartella `sorgenti`, e di scrivere l'output nella cartella `raccolte`.

2.2 Dettagli e opzioni

Il codice del programma è diviso su quattro file:

- `q_gen.py`
- `gen_tools.py`
- `combinatorics.py`
- `verify.py`

I quattro file devono essere tutti nella stessa cartella in cui si lavora. Nella stessa cartella deve essere presente anche il file di configurazione `config.json` (al momento non è possibile cambiare il nome del file di configurazione).

Il file `config.json` è un file in formato `JSON`. Nell'appendice A è brevemente riepilogata la sintassi di questo formato. Nel paragrafo 4 è spiegata nel dettaglio la struttura del file `config.json`.

Il programma si esegue da un terminale, spostandosi nella cartella in cui si trova il programma, con il comando

```
python3 q_gen.py
```

Richiamato in questo modo, elabora i file specificati nella lista in `"exec"` del file `config.json` (si veda il paragrafo 4). Se non ce ne sono, il programma chiede in input il nome o il percorso di un file.

Chiamando il programma con il comando

```
python3 q_gen.py -d <nome_cartella>
```

viene ignorato il contenuto del file `config.json` e vengono elaborati tutti i file presenti nella cartella `<nome_cartella>`. Se non si specifica `<nome_cartella>`, cioè dando il comando:

```
python3 q-gen.py -d
```

viene usata la cartella di input di default, specificata nel file `config.json`, ovvero `sorgenti`.

Il programma può anche essere chiamato con l'opzione `-i` ovvero `--input` e il nome del file, ad esempio

```
python3 q-gen.py -i <file_json_da_elaborare>
```

In questo caso viene ignorato il campo "exec" del file `config.json`.

Il nome dei file può essere dato in input

- senza prefisso, senza estensione `.json` e senza specificare tutto il percorso (in questo caso viene usato il prefisso specificato in `config.json` e viene cercato il file nella cartella specificata in `config.json`; si veda ancora il paragrafo 4);
- il file può essere invece specificato con il percorso completo o relativo alla cartella di lavoro (con o senza estensione `.json`): in questo caso non viene aggiunto il prefisso, né viene usata la cartella di input configurata in `config.json`, ma viene aggiunta l'estensione `.json` se necessario.

All'inizio dell'elaborazione il software controlla:

- che esista il file `config.json` e che sia corretto il suo formato,
- che esistano tutti i template necessari (specificati nel file `config.json`)
- che ci sia ciascun file indicato come file da elaborare,
- che sia corretto il formato `JSON` di ciascun file da elaborare,
- che ci siano i campi necessari (definiti nel file `config.json`) in ciascun file da elaborare
- che ciascun file da elaborare sia coerente (ad es. che in una MCQ la somma delle opzioni scelte sia esattamente 100).

Se una di queste verifiche fallisce, l'esecuzione termina con un messaggio di errore (in alcuni casi viene dato solo un avviso e l'elaborazione continua perché il problema non è bloccante).

Dopo le verifiche, vengono generate le combinazioni e prodotta la raccolta di domande. La raccolta viene salvata nella cartella specificata in `config.json` e al nome del file viene aggiunto il prefisso definito. Se non è specificata una cartella di output, il file viene salvato nella cartella corrente.

2.3 Concatenazione

Il programma produce un file XML per ogni file in input. È utile importarli in moodle uno per volta per verificare la correttezza delle domande. Però se si vuole rendere più veloce l'importazione, si può utilizzare l'opzione di concatenazione offerta dal programma `q-gen.py`.

Con l'opzione `-c` o `--concat` si istruisce il programma a concatenare tutte le raccolte di domande che sono nella cartella specificata:

```
python3 q-gen.py -c<nome_cartella>
```

Il file di output si chiama `000_tutte.xml` e viene generato nella stessa cartella `<nome_cartella>`.

Attenzione: i file delle domande devono essere già stati generati.

Se il file `000_tutte.xml` si trova già nella cartella **raccolte**, non viene concatenato agli altri file e viene sovrascritto, se non gli è stato cambiato nome (o non è stata cambiata la definizione del nome nel file `config.json`).

3 Formato dei file di input

I file di input devono essere in formato JSON.

3.1 Chiavi per tutti i tipi di domande

"categoria"

Obbligatoria. La categoria in cui devono essere inserite le domande su Moodle

"nome_domanda"

Il nome delle domande: quelle che vengono da un unico file avranno lo stesso nome

"tipo_domanda"

Il tipo di domanda. Come valori della chiave **"tipo_domanda"** si possono specificare i seguenti:

- **"cloze"**: tipo cloze
- **"dd"**: tipo drag & drop
- **"mcq"**: tipo multiple choice question.

"frasi"

Obbligatoria. Le affermazioni, o nel caso di MCQ la domanda e le possibili risposte.

Il valore della chiave "frasi" è un dizionario e ha chiavi "1","2","3",... (numeri tra virgolette a partire da 1 e consecutivi). Ogni numero identifica un gruppo di affermazioni e il valore della chiave è una lista. Ogni affermazione è a sua volta un dizionario con chiavi "frase" e "corretta". Quindi:

```
"frasi"{  
  "1": [{"frase": "affermazione1", "corretta": "risposta1"},  
        {"frase": "affermazione2", "corretta": "risposta2"}],  
  "2": [{"frase": "affermazione3", "corretta": "risposta3"}]  
}
```

Per il significato del campo "corretta", si veda "risposte" nella sezione 3.5.

"numero_di_frase"

Obbligatoria. Il numero di affermazioni per domanda, da scegliere tra le "frasi". Nel caso delle "mcq", "numero_di_frase" è il numero delle possibili risposte tra cui scegliere più una. Questo perché anche la domanda in una "mcq" viene scelta a caso tra un gruppo di domande, quelle del gruppo "1".

"scelte"

Obbligatoria per le domande "mcq"; opzionale per gli altri tipi di domande. Si veda la sezione 3.7

3.2 Chiavi per le domande "dd"

Sono obbligatorie, in aggiunta alle chiavi della sezione 3.1, le chiavi

"risposte"

Obbligatoria. Sono le scelte che si possono spostare (drag&drop) per rispondere. Si vedano i dettagli nella sezione 3.5.

"consegna"

Obbligatoria. La consegna: rispondete..., completate... ecc.

"spazio_risposta"

Obbligatoria. La stringa che si scriverà al posto del buco da riempire; non deve comparire altrove nel file JSON.

"se_infinite"

Opzionale. Specifica se le scelte drag&drop si esauriscono o sono sempre disponibili; se non lo si usa, si intende che le scelte non sono infinite

3.3 Chiavi per le domande "cloze"

In aggiunta alle chiavi della sezione 3.1 i campi

"tipo_di_cloze"

Obbligatoria. Al momento il valore può essere

- "MULTICHOICE_H",
- "MULTICHOICE_V",
- "MULTICHOICE_VS",
- "MULTICHOICE_HS",
- "SHORTANSWER"

"risposte"

Obbligatoria. Specifica le scelte possibili o le risposte accettate. Si vedano i dettagli nella sezione 3.5.

"consegna"

La consegna: rispondete..., completate... ecc.

3.4 Chiavi per le domande "mcq"

Sono obbligatori in aggiunta alle etichette della sezione 3.1 i campi

"frazioni_gruppi"

Obbligatorio. Il valore è un dizionario che specifica per ogni gruppo di "frasi" quanto vale la risposta: 100, 0, o una frazione, come usata da Moodle. Si vedano i dettagli nella sezione 3.6.

3.5 Dettagli sul campo "risposte"

Il campo "risposte" è sempre un dizionario. Ci sono alcuni vincoli sulle etichette:

- per le domande cloze "MULTICHOICE", le etichette possono essere arbitrarie; quindi ad esempio

```
"risposte": {"1": "rosso", "2": "verde", "3": "giallo"}
```

se si vuole che le tre opzioni siano rosso, verde e giallo

- per le domande cloze **SHORTANSWER** le etichette devono essere una per ciascun gruppo di "frasi" e quindi "1","2","3",...; il valore associato ha esattamente il formato che va scritto in una cloze **SHORTANSWER**. Ad esempio:

```
"risposte":{  
  "1": "{1:SHORTANSWER:=rosso}",  
  "2": "{1:SHORTANSWER:=giallo}",  
  "3": "{1:SHORTANSWER:=rosso}"  
}
```

- per le domande drag&drop le etichette devono essere numeri consecutivi, da "1" in avanti. Esempio:

```
"risposte":{"1": "rosso", "2": "verde", "3": "blu", "4": "giallo"}
```

Il campo "corretta" nei dizionari che rappresentano le affermazioni (sezione 3.1) contiene la chiave di "risposte" il cui valore è la risposta corretta per quella frase. Quindi se

```
"risposte" : {"1": "rosso", "2": "verde", "3": "blu", "4": "giallo"}
```

e

```
"frasi"{  
  "1": [{"frase": "frase1", "corretta": "2"},  
        {"frase": "frase2", "corretta": "4"}],  
  "2": [{"frase": "frase3", "corretta": "1"}]  
}
```

la risposta corretta per la frase1 è verde, per la frase2 è giallo e per la frase3 è rosso.

Nel caso delle **SHORTANSWER** per comodità ogni gruppo può essere una lista ma in quel caso ogni affermazione di ciascun gruppo deve avere la stessa **SHORTANSWER**; la chiave della **SHORTANSWER** corrispondente è la chiave del gruppo. Ad esempio si può scrivere:

```
"frasi":{
  "1":["frase1","frase2"],
  "2":["frase3"]
}
```

che viene interpretato come

```
"frasi":{
  "1":[{"frase":"frase1", "corretta":"1"},
      {"frase":"frase2", "corretta":"1"}],
  "2":[{"frase":"frase3", "corretta":"2"}]
}
```

3.6 Dettagli sul campo "frazioni_gruppi"

Il campo "frazioni_gruppi" è un dizionario. Qui, per ogni gruppo è specificato il valore della risposta, tranne che per il gruppo "1" che contiene le possibili domande. I valori, come previsto da Moodle sono ad esempio "100", "50", "0", "-50", "-3,33333" (sempre scritti tra virgolette, si può usare indifferentemente il punto o la virgola per separare i decimali). Esempio:

```
"frazioni_gruppi":{"2":"100","3":"0","4":"50","6":"-50"}
```

Il fatto che manchi la chiave "5", significa "5":"0". A tutte le frasi di ciascun gruppo corrisponde quindi la stessa frazione, e di conseguenza le affermazioni devono essere raggruppate in base al loro valore di verità (vere insieme, false insieme). È responsabilità di chi progetta le domande scegliere le frasi in modo che tutto abbia senso, ma il programma fa alcune elementari verifiche. In proposito, si veda il campo "scelte" nel paragrafo 3.7.

3.7 Dettagli sul campo "scelte"

Il campo "scelte", se presente, è una lista di liste e specifica come vengono scelti i quesiti dai vari gruppi in "frasi".

Ad esempio:

```
"scelte":[[1,2,1],[1,1,1,1]]
```

Ognuna delle liste contenute deve avere lunghezza pari al massimo al numero dei gruppi. Se i gruppi sono 3, [1,2,1] significa che il programma deve scegliere 1 affermazione dal gruppo "1", 2 dal "2" e 1 dal "3".

Se i gruppi sono ad esempio 4, [1,2,1] equivale a [1,2,1,0]. Se ci sono più liste in "scelte", per ciascuna lista vengono scelte le combinazioni come specificato.

Per ciascuna delle liste contenute in "scelte", la somma dei valori deve essere pari a "numero_di_frase". Quindi se "numero_di_frase": "4" e i gruppi sono 5, la lista [1,2,1,0,0] va bene ma la lista [2,1,0,2,2] no perché i suoi elementi hanno somma 7.

Se il campo "scelte" manca o è vuoto ("choices": []) il programma sceglie tutte le possibili combinazioni: attenzione che potrebbero essere tante, perfino troppe.

3.7.1 "scelte" per "mcq"

Nel caso di "mcq" il campo "scelte" è obbligatorio e ogni lista deve iniziare con 1 (ad esempio [1,0,2,1] ma non [2,0,1,1]), perché ci deve essere la domanda e ce ne deve essere una sola.

Se la lista specifica una risposta con "group_fraction" 100, automaticamente il programma decide che si tratta di una MCQ con una sola alternativa. Altrimenti, viene creata una MCQ con più alternative. Si può forzare una domanda che ha una risposta con frazione 100% ad accettare risposte multiple con l'opzione "risposte_multiple" (si vedano le opzioni nel paragrafo 3.8).

Nel caso di una "mcq", il software controlla inoltre, per ogni lista in "scelte", che la somma delle "frazioni_gruppi" positive sia esattamente 100. Ad esempio, va bene:

```
"scelte": [[1,1,2], [1,1,1,1]]  
"frazioni_gruppi": {"2": "100", "3": "0", "4": "0"}
```

mentre, nell'esempio successivo, nessuna delle "scelte" è accettabile:

```
"scelte": [[1,2,1], [1,0,2,1]]  
"frazioni_gruppi": {"2": "100", "3": "0", "4": "50"}
```

Inoltre, se si tratta di una domanda che ammette risposte multiple, il programma controlla che non ci siano risposte con frazione 0; in questo caso dà solo un avviso ma continua l'elaborazione. Quindi in questo caso sarà dato un avviso per la seconda lista ma non per la prima:

```
"scelte": [[1,1,2], [1,0,1,2]]  
"frazioni_gruppi": {"2": "100", "3": "0", "4": "50"}
```

Il campo "scelte" è obbligatorio per le "mcq" perché le scelte devono essere sotto il controllo dell'utente.

3.8 Opzioni: campo "opzioni"

Il campo "opzioni" è opzionale. È una lista. Le opzioni possibili sono:

"moodle_ordine_casuale"

Istruisce il programma a indicare "Alternative in ordine casuale" per queste domande. Ha senso per "mcq", "dd" e "cloze" di tipo "MULTICHOICE".

"risposte_multiple"

Ha senso per domande "mcq" e istruisce il programma a definire tutte le domande come domande con più risposte possibili, anche quelle che hanno una sola risposta giusta, con frazione "100".

"elenco_puntato"

Presenta i quesiti come un elenco puntato. Ha senso per "dd" e "cloze".

"permutazioni"

Richiede di scrivere le combinazioni in tutti gli ordini possibili; per chiarire bene questo, entriamo nel dettaglio nel paragrafo 3.8.1.

3.8.1 Combinazioni e "permutazioni"

Il programma calcola tutte le possibili combinazioni delle frasi secondo i requisiti specificati in "scelte", ma senza permutazioni; cioè se viene scelta la combinazione di frasi A,B,C non verranno scelte A,C,B né B,C,A, né B,A,C ecc. Il programma sceglie solo una delle permutazioni, a caso; non sceglie necessariamente A,B,C; magari sceglie C,B,A, ma solo quella.

Se invece si usa l'opzione "permutazioni", allora il programma produrrà una domanda per ogni permutazione. Nel nostro esempio, 6 domande con le stesse frasi ma in ordine diverso: A,B,C e A,C,B e B,A,C e B,C,A e C,A,B e C,B,A. Sostanzialmente simula le "Alternative in ordine casuale" di una domanda MCQ.

Ha senso per domande "dd" e "cloze".

Si noti che l'opzione "moodle_ordine_casuale" riordina le opzioni drag&drop per le domande "dd", non le frasi. Per le domande "cloze" "MULTICHOICE", le versioni con la 'S' in fondo ("MULTICHOICE_VS" e "MULTICHOICE_HS") analogamente riordinano le opzioni per le risposte.

4 File di configurazione

Nella cartella in cui si trova (e si esegue) il programma, deve essere presente il file `config.json` che contiene parametri di configurazione.

Le chiavi del file `config.json` non possono mai essere modificate, altrimenti sarebbe necessario modificare il codice del programma.

4.1 Parametri utili all'utente

I valori delle chiavi descritte in questa sezione possono essere sempre modificati tranquillamente, curando la coerenza con file e directory.

"exec"

È una lista di nomi di file eventualmente vuota cioè `[]`; se ci sono delle stringhe, vengono interpretate come nomi di file su cui eseguire il programma. Se il programma viene lanciato con delle opzioni (si veda il paragrafo 2) "exec" viene ignorato.

"database"

È una lista di nomi di file; se in "exec" è scritto "ALL", viene eseguito il programma su i file elencati in .

"filenames"

È un dizionario che contiene nomi di directory e prefissi di file. In particolare

- "source_dir" e "out_dir" specificano le cartelle (contenute nella stessa cartella del programma) rispettivamente da cui il programma deve leggere i file di input e in cui deve scrivere l'output;
- "sourcefile_prefix" e "outfile_prefix" specificano il prefisso che deve avere ciascun file rispettivamente di input e di output. Ad esempio può essere comodo non usare cartelle separate per input e output e invece distinguere i file di input e out dal prefisso, come ad esempio `sorg_antigone.json` e `raccolta_antigone.xml`.
- "collections_concat" definisce il nome del file di output del programma usato con l'opzione `-c/--concat`.

Tutti i valori di queste chiavi, tranne quello di "collections_concat", possono essere impostati a "" cioè essere vuoti.

Come spiegato nella sezione 2, il programma è molto flessibile riguardo al modo in cui sono scritti i nomi dei file in input in "exec" e "database".

4.2 Altri parametri

I valori delle chiavi elencate in questa sezione possono essere modificati ma mantenere la coerenza può essere meno ovvio (per i template) o più laborioso (per etichette e opzioni).

"templates"

È un dizionario e definisce la cartella e i nomi dei file in cui sono definiti gli schemi da cui vengono composti i file di domande nel formato Moodle XML. Si possono modificare, ma bisogna anche cambiare nome della cartella e nomi dei file.

"necessary_input_fields"

È un dizionario. Elenca i campi necessari nei file JSON in input: "all" vale per tutti i tipi di domande, "dd", "mcq" e "cloze" per i singoli tipi; invece "none" elenca campi che sono opzionali. Questo dizionario viene usato per controllare la coerenza del file da elaborare, e per leggere correttamente i file in input.

Se si desidera modificare il nome di un campo nei file di input (ad es. "tipo_di_domanda" invece che "tipo_domanda"), si può cambiare il valore a fianco di "tipo_domanda", cioè "question_type":"tipo_di_domanda". Non si può cambiare alcuna chiave dei dizionari. Naturalmente poi in tutti i file in input si deve scrivere "tipo_di_domanda" invece che "tipo_domanda".

"meaningful_options"

È un dizionario. Viene usato per verificare che le opzioni nel file di input esistano e siano coerenti con il tipo di domanda. Viene usato anche per leggere correttamente i file in input. Si possono cambiare i valori, se si desidera chiamare diversamente le opzioni, come discusso per "necessary_input_fields". Mai cambiare le etichette dei dizionari.

"cloze_types"

Elenca i tipi cloze gestiti dal programma. Non si può modificare.

A Formato dei file JSON

Un file JSON è un *dizionario*, ovvero ha questo formato:

```
{  
  "chiave1":valore1,  
  "chiave2":valore2,  
  ...  
  "chiaveN":valoreN  
}
```

In particolare, il contenuto del file è tra parentesi graffe e il file è una sequenza di *coppie chiave-valore*, separate da virgola. L'ultima coppia non termina con una virgola.

Ogni *chiave* è una stringa (una parola tra virgolette o apici). Ogni *valore* può essere una stringa (ad es. "ciao") oppure un altro dizionario oppure una *lista* (si veda dopo).

Se ad esempio la coppia *chiave-valore* (o *elemento*) di un dizionario è

```
"colore": "rosso"
```

si dice che "rosso" è *il valore della chiave "colore"*.

Una *lista* è una sequenza di valori tra parentesi quadre, separati da virgola:

```
[valore1, valore2, ..., valoreN].
```

(l'ultimo valore non è seguito da virgola). Gli elementi di una lista possono essere a loro volta dizionari, liste, stringhe o numeri.

Commenti in un file JSON

Non esiste un formato specifico per aggiungere commenti in un file JSON.

Ma ad un file JSON si possono aggiungere chiavi a piacere. Nel nostro caso, se la chiave non è specificata in questo manuale, viene semplicemente ignorata dal programma `q_gen`.

Quindi, per inserire un commento in un file JSON, è sufficiente inserire una nuova coppia chiave-valore nel dizionario, con una chiave nuova. Ad esempio

```
"NOTA": "Il gruppo '2' è il più importante."
```

Il campo "NOTA" verrà ignorato.

Si noti che, nel valore di "NOTA", intorno a 2 sono usati gli apici e non le virgolette. Le virgolette avrebbero l'effetto di terminare anzitempo il valore della chiave "NOTA".

B Nomi delle chiavi dei file in input

Le chiavi che compaiono nei file JSON in input al programma possono essere modificate tramite il file `config.json`. La tabella 1 mostra a destra i nomi delle chiavi al momento della stesura di questo manuale.

Tabella 1: Elenco delle definizioni attualmente impostate nel file `config.json`

Chiavi in <code>config.json</code> , non modificabili	Valori impostati, modificabili
"source_dir"	"sorgenti"

Continued on next page

Tabella 1: Elenco delle definizioni attualmente impostate nel file `config.json`
(Continued)

Chiavi in config.json, non modificabili	Valori impostati, modificabili
"out_dir"	"raccolte"
"sourcefile_prefix"	""
"outfile_prefix"	"raccolta_"
"collections_concat"	"000_tutte"
"category"	"categoria"
"question_name"	"nome_domanda"
"question_type"	"tipo_domanda"
"number_of_statements"	"numero_di_frase"
"statements"	"frasi"
"risposte"	"risposte"
"task"	"consegna"
"answers_placeholder"	"spazio_risposta"
"cloze_type"	"tipo_di_cloze"
"group_fractions"	"frazioni_gruppi"
"choices"	"scelte"
"computed_choices"	"scelte_calcolate"
"options"	"opzioni"
"statement"	"frase"
"corretta"	"corretta"
"if_infinite"	"se_infinite"
"multiple"	"risposte_multiple"
"moodle_shuffle"	"moodle_ordine_casuale"
"itemized"	"elenco_puntato"
"all_orders"	"permutazioni"